

C++ Physical Computing on the Raspberry Pi 5

Answer Key — wiringPi Edition

Companion to *C++ wiringPi Edition.pdf*. Every blank filled, with the reasoning behind each choice — not just the value. Read a lesson's explanation only after you've attempted it; if you're peeking before writing anything, that's the shortcut costing you the learning.

Lesson 1 — Blink an LED

```
#include <wiringPi.h>
#include <iostream>

constexpr int LED = 17;

int main() {
    if (wiringPiSetupGpio() == -1) {
        std::cerr << "wiringPi init failed\n";
        return 1;
    }
    pinMode(LED, OUTPUT);

    for (int i = 0; i < 10; ++i) {
        digitalWrite(LED, HIGH);
        delay(500);
        digitalWrite(LED, LOW);
        delay(500);
    }
    return 0;
}
```

Why these values. `OUTPUT` because you're driving the LED, not reading it. `500` ms on each side gives a clean 1 Hz blink (500 on + 500 off = 1000 ms period) — visible and unhurried; anything under ~30ms per phase and your eye starts averaging it into a dim steady glow rather than a blink (that's the PWM principle you'll hit in Lesson 4, arriving early). `10` iterations is arbitrary — the point was picking *some* finite count instead of `while(true)` so the program actually terminates for review.

Predict-first answer: the minimum requirement is that the pin is *distinctly* HIGH and LOW for long enough for your eye to register each state — roughly 30–50 ms minimum, comfortably above 100 ms. Delete both `delay()` calls and the loop still toggles correctly in logic, but it runs at whatever speed the CPU allows (likely tens of thousands of toggles/second) — the LED appears to glow at a dim, steady brightness because your eye can't track individual switches. That's PWM by accident, and it's the exact mechanism Lesson 4 formalizes on purpose.

SOS extension, worked:

```
void flash(int ms) {
    digitalWrite(LED, HIGH);
    delay(ms);
    digitalWrite(LED, LOW);
    delay(200);           // gap between symbols within a letter
}

void dot() { flash(200); }
void dash() { flash(600); }

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    pinMode(LED, OUTPUT);

    dot(); dot(); dot();    delay(400);    // S -- extra gap between letters
    dash(); dash(); dash(); delay(400);    // O
    dot(); dot(); dot();    // S
}
```

The two named helpers (`dot / dash`) rather than one `flash(int)` called with different arguments everywhere makes the `main()` sequence read like actual Morse — self-documenting. The **letter-gap delay** (400ms, longer than the 200ms inter-symbol gap already inside `flash`) is what makes S-O-S parse as three distinct letters instead of nine identical blinks; without it a human eye can't tell where one letter ends and the next begins.

Lesson 2 — Read a button

```
#include <wiringPi.h>
#include <iostream>

constexpr int BTN = 2;

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    pinMode(BTN, INPUT);
    pullUpDnControl(BTN, PUD_UP);

    int prev = HIGH;           // resting level with a pull-up
    while (true) {
        int now = digitalRead(BTN);
        if (now == LOW && prev == HIGH) { // press edge: HIGH -> LOW
            std::cout << "Pressed!\n";
        }
        prev = now;
        delay(5);
    }
}
```

Predict-first answer: with a pull-up, the pin is pulled to **HIGH** (3.3V) when the button is *open*. Pressing the button connects the pin to GND, pulling it to **LOW**. So: **unpressed = HIGH, pressed = LOW** — the electrical opposite of what "pressed = on" intuition suggests. This inversion is the single most common source of "my button logic is backwards" bugs.

Why **PUD_UP and not **PUD_DOWN** :** the circuit is wired button→GND, meaning the button only ever pulls the pin *down* when pressed. If nothing pulls it up when the button is open, the pin floats — reads unpredictable noise. **PUD_UP** supplies that missing pull, giving a clean, defined **HIGH** at rest.

Why **now == LOW && prev == HIGH :** an edge is a *transition*, not a level. Checking **now == LOW** alone would fire the print continuously for every 5ms poll while the button stays held — you'd get a flood of prints, not one. Requiring the *previous* reading to have been **HIGH** means the print fires exactly once, on the single poll cycle where the level changed.

Lesson 3 — Button-controlled LED

```
#include <wiringPi.h>

constexpr int BTN = 2;
constexpr int LED = 17;

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    pinMode(BTN, INPUT);
    pullUpDnControl(BTN, PUD_UP);
    pinMode(LED, OUTPUT);

    bool led_on = false;
    int prev = HIGH;
    while (true) {
        int now = digitalRead(BTN);
        if (now == LOW && prev == HIGH) { // press edge
            led_on = !led_on;           // TOGGLE
            digitalWrite(LED, led_on ? HIGH : LOW);
        }
        prev = now;
        delay(5);
    }
}
```

Predict-first answer: libgpiod's grouped **request** buys you one thing wiringPi's independent **pinMode** calls don't give automatically: a single object whose lifetime governs *both* lines together, and one atomic multi-line update call if you needed to change several pins in lockstep. For this circuit it doesn't matter — the two pins are logically independent (one read, one write) and nothing here requires atomic simultaneous changes. The cost only shows up in circuits where several output pins must change *together* in the same instant (e.g. a parallel bus) — not the case here.

The toggle itself: **led_on = !led_on** is the whole state machine. Because **led_on** persists across loop iterations (it's declared once, outside the loop), each press edge flips it exactly once — press 1 turns the LED on, press 2 turns it off, etc.

Lesson 4 — PWM: LED brightness

```
#include <wiringPi.h>
#include <softPwm.h>

constexpr int LED = 18;

int main() {
    if (wiringPiSetupGpio() == -1) return 1;

    softPwmCreate(LED, 0, 100);           // range 100 -> duty is a direct percentage
    while (true) {
        for (int duty = 0; duty <= 100; ++duty) {
            softPwmWrite(LED, duty);
            delay(15);
        }
        for (int duty = 100; duty >= 0; --duty) {
            softPwmWrite(LED, duty);
            delay(15);
        }
    }
}
```

Hardware PWM version:

```
pinMode(18, PWM_OUTPUT);
pwmSetMode(PWM_MODE_MS);
pwmSetRange(1024);
pwmSetClock(188);           // ~19.2MHz / (1024 * 100Hz) ≈ 187.5 -> 188
pwmWrite(18, 512);          // 512/1024 = 50%
```

Predict-first answer: with range 100, duty **50** gives ~50% brightness — the range *is* the percentage scale, so the two numbers line up directly. `softPwm` runs a background thread per pin that toggles the pin in software at whatever rate the OS scheduler allows — inherently jittery because the OS can preempt that thread at any time. `pwmWrite` drives a **hardware** PWM peripheral in RP1 silicon that free-runs independent of the CPU scheduler — the toggling happens in hardware, so the signal is rock-steady regardless of what else the Pi is doing.

The clock math: the Pi's PWM base clock is 19.2MHz. `pwmSetRange(1024)` means one full PWM cycle takes 1024 clock ticks. Dividing the base clock by the divisor you set with `pwmSetClock()` gives your *tick* rate; dividing that by the range gives your *PWM frequency*. Solving for a divisor that lands near 100Hz: $19,200,000 / (1024 \times 100) \approx 187.5$, rounded to **188**. Try the arithmetic yourself for a different target frequency (say, 1kHz) before trusting this number blindly — it's exactly the kind of thing worth deriving, not memorizing.

Why duty is 0..1024 here but 0..100 for softPwm: each is scaled to whatever range you set in its respective `*Range` / `*Create` call. There's no fixed convention — read the range you configured before writing a duty value, every time.

Lesson 5 — RGB LED

```
#include <wiringPi.h>
#include <softPwm.h>

constexpr int R = 17, G = 27, B = 22;

struct Color { int r, g, b; };

void show(const Color& c) {
    softPwmWrite(R, c.r);
    softPwmWrite(G, c.g);
    softPwmWrite(B, c.b);
}

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    softPwmCreate(R, 0, 100);
    softPwmCreate(G, 0, 100);
    softPwmCreate(B, 0, 100);

    Color magenta{100, 0, 100};
    while (true) show(magenta);
}
```

Predict-first answer: hardware PWM on the Pi 5 exposes only **two** independent channels total (BCM 12 and 18 share channel 0; 13 and 19 share channel 1) — you cannot get three independent hardware-driven duty cycles simultaneously no matter which three pins you pick, because at most two of them can be on different channels. Three colors *requires* software PWM, where each `softPwmCreate` spins up its own independent thread with no such channel-sharing limit.

Magenta = {100, 0, 100}: full red, no green, full blue — matches the RGB definition given in the predict prompt directly.

Lesson 6 — 7-segment display (5161BS)

```
#include <wiringPi.h>
#include <array>
#include <cstdlib>

// segment order: a b c d e f g (bit 0 = a ... bit 6 = g), 1 = lit
const std::array<uint8_t, 10> DIGIT = {
    0b0111111, // 0: a b c d e f on, g off
    0b0000110, // 1: b c on
    0b1011011, // 2: a b d e g on
    0b1001111, // 3: a b c d g on
    0b1100110, // 4: b c f g on
    0b1101101, // 5: a c d f g on
    0b111101, // 6: a c d e f g on
    0b0000111, // 7: a b c on
    0b1111111, // 8: all on
    0b1101111, // 9: a b c d f g on
};

const int SEG[7] = {21, 20, 16, 22, 23, 24, 12};

void write_digit(uint8_t pattern) {
    for (int s = 0; s < 7; ++s) {
        bool on = (pattern >> s) & 1;
        digitalWrite(SEG[s], on ? HIGH : LOW);
    }
}

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    for (int s = 0; s < 7; ++s) pinMode(SEG[s], OUTPUT);
    for (int d = 0; d <= 9; ++d) {
        write_digit(DIGIT[d]);
        delay(800);
    }
}
```

Predict-first answer: '3' lights a, b, c, d, g (5 segments; skips e, f). '7' lights a, b, c only (3 segments — the minimal "checkmark" 7, no bottom serif). Total pins needed: **7** for the segments alone (8 if the display has a separate decimal-point segment you're also driving).

Reading the bit extraction: `(pattern >> s) & 1` shifts the pattern right by `s` bits, moving the bit you want down to position 0, then masks everything else off with `& 1`. For `s=0` this reads segment a (the least significant bit as defined in this table); for `s=6` it reads segment g. This is the same shift-and-mask idiom you'll reuse for the MPU6050 (Lesson 10) and RGB565 packing (Lesson 12) — worth internalizing here where the stakes are just an LED segment, not a sensor reading.

std::array vs. raw bitmask, for your review defense: `std::array<uint8_t,10>` gives you compile-time bounds checking with `.at()`, a known `.size()`, and it's a real C++ container (works with range-for, iterators, algorithms). A raw `uint8_t DIGIT[10]` is marginally lighter-weight but gives up all of that for no real gain here — the array is tiny and read-only, so the safety costs nothing.

Check your own wiring before trusting this table. The exact bit-to-segment mapping depends on which physical pin you wired to which segment letter, and whether your unit is common-cathode (this table assumes common-cathode: 1 = lit) or common-anode (inverted: 0 = lit). Cross-check against [5161bs.pdf](#) rather than assuming this table matches your breadboard.

Lesson 7 — Shift register (74HC595)

```
#include <wiringPi.h>
#include <cstdint>

constexpr int DATA = 17, CLOCK = 27, LATCH = 22;

void shiftOut595(uint8_t value) {
    for (int i = 7; i >= 0; --i) {
        bool bit = (value >> i) & 1;
        digitalWrite(DATA, bit ? HIGH : LOW);
        digitalWrite(CLOCK, HIGH);           // rising edge shifts the bit in
        digitalWrite(CLOCK, LOW);
    }
    digitalWrite(LATCH, HIGH);               // rising edge copies shift reg -> output reg
    digitalWrite(LATCH, LOW);
}
```

Predict-first answer: looping `i` from 7 down to 0 sends the **most significant bit first**. The 74HC595 shifts each new bit into position 0 of its internal register and pushes everything else up by one — so the *first* bit you send ends up furthest from the input by the time all 8 are in, landing on the *last* output (Q7/LED8). Shift out `0b10000000` (only the MSB set) starting from `i=7`: that 1-bit goes in first, gets pushed all the way to the far end — the **last** LED lights. Reverse the loop direction (`i` from 0 to 7, LSB first) and the same byte would light the **first** LED instead. Bit order and loop direction are two names for the same choice.

Why clock and latch each get HIGH then LOW: these chips are edge-triggered, not level-triggered — the *transition* is what causes the internal action (shift, or latch), not the level being held. Pulsing HIGH-then-LOW guarantees one clean rising edge each time through the loop, which is exactly one shift per bit. The same rising-edge logic then latches the whole assembled byte to the output pins in one action.

Lesson 8 — I²C: analog input via ADC

```
#include <wiringPiI2C.h>
#include <unistd.h>
#include <cstdint>

class I2C {
    int fd;
public:
    explicit I2C(int addr) {
        fd = wiringPiI2CSetup(addr);
    }
    ~I2C() { close(fd); }

    int readByte()          { return wiringPiI2CRead(fd); }
    int writeByte(uint8_t v) { return wiringPiI2CWrite(fd, v); }
    int readReg8(uint8_t reg) { return wiringPiI2CReadReg8(fd, reg); }
};

int main() {
    I2C adc(0x48);                // typical ADS7830 address; confirm with i2cdetect
    while (true) {
        adc.writeByte(0x84);      // ADS7830: single-ended, channel 0, internal ref on
        int value = adc.readByte(); // 0..255
        // print value ...
    }
}
```

Predict-first answer: the ADC returns a **number** — a digital code (here, 0–255 for an 8-bit converter) proportional to the input voltage, not the voltage itself. Converting that code back to volts is a separate calculation you do in software (`code / 255.0 * Vref`), using the chip's known reference voltage from its datasheet.

Why `close(fd)` belongs in the destructor: `wiringPiI2CSetup` calls `open()` under the hood and hands you back a raw file descriptor — an OS-level handle to `/dev/i2c-1`. File descriptors are a limited resource; if you keep constructing `I2C` objects without ever closing the old ones, you'll eventually exhaust the process's descriptor table. Putting `close(fd)` in `~I2C()` means it happens automatically whenever an `I2C` object goes out of scope — the exact RAII discipline the base guide calls out as the one place wiringPi still rewards it.

The command byte `0x84`: ADS7830's command byte packs the input mode and channel select into specific bit fields — `0x84` selects single-ended mode on channel 0 with the internal reference enabled, per its datasheet. If you're using the PCF8591 instead, its command byte format differs — check [PCF8591_datasheet.pdf](#) rather than reusing this value.

Lesson 9 — I²C I/O expander (PCF8574) + LCD1602

```
void lcd_send(I2C& lcd, uint8_t value, bool is_data) {
    uint8_t hi = value & 0xF0;
    uint8_t lo = (value & 0x0F) << 4;    // low nibble, shifted into the high position
    lcd_write4(lcd, hi, is_data);
    lcd_write4(lcd, lo, is_data);
}

void lcd_init(I2C& lcd) {
    delay(50);                          // wait for LCD power-on (datasheet: >40ms after
Vcc)
    lcd_write4(lcd, 0x30, false); delay(5);
    lcd_write4(lcd, 0x30, false); delayMicroseconds(150);
    lcd_write4(lcd, 0x30, false);
    lcd_write4(lcd, 0x20, false);        // switch to 4-bit mode
    lcd_send(lcd, 0x28, false);          // function set: 4-bit, 2-line, 5x8 font
    lcd_send(lcd, 0x0C, false);          // display on, cursor off, blink off
    lcd_send(lcd, 0x06, false);          // entry mode: increment, no shift
    lcd_send(lcd, 0x01, false); delay(2); // clear display (needs extra time)
}
```

Predict-first answer: skip a delay and the LCD's internal controller — which is busy processing the previous instruction on its own slow internal timescale — receives the next command before it's ready. The controller either drops the new command, or worse, misinterprets a fragment of it as something else. Symptom: garbled characters, or nothing displays at all, with code that "looks correct" on inspection. This is a purely timing-dependent bug, invisible from reading the C++ alone.

Why the low nibble gets shifted left by 4: the PCF8574 backpack drives the LCD in 4-bit mode, sending one nibble (4 bits) per bus write rather than a full byte. The LCD's hardware always expects the active nibble in the *upper* 4 bits of the data line, regardless of whether you're sending the value's high or low nibble — so the low nibble has to be shifted up to occupy that same upper position before it's sent.

The specific init sequence (three `0x30` writes, then `0x20`) is the HD44780 controller's documented "reset to a known state" handshake — it works regardless of whatever state the display happened to power on in. Don't reorder or shortcut it; it's copied near-verbatim from every HD44780 datasheet for a reason.

Lesson 10 — MPU6050 accelerometer + gyro

```
int16_t axis(uint8_t hi, uint8_t lo) {
    return (int16_t)( (hi << 8) | lo );
}

int main() {
    I2C imu(0x68);
    imu.writeReg8(0x6B, 0x00);           // wake the device (PWR_MGMT_1)

    while (true) {
        uint8_t xh = imu.readReg8(0x3B), xl = imu.readReg8(0x3C);
        int16_t ax = axis(xh, xl);
        double g = ax / 16384.0;         // sensitivity for +/-2g range
        // print all three axes ...
    }
}
```

Predict-first answer: forget the sign and you'd cast/store the 16-bit combination as `uint16_t` (or worse, mask it into a smaller unsigned type) — any negative acceleration (tilted the "wrong" way) would read back as a huge positive number instead of a small negative one, because two's-complement negative values only make sense when the type doing the interpreting is itself signed.

Why `hi << 8 | lo` and not the reverse: the register map documents `ACCEL_XOUT_H` (0x3B) as the *high* byte and `ACCEL_XOUT_L` (0x3C) as the *low* byte of the same 16-bit value. Shifting `hi` left by 8 bits moves it into the upper half of a 16-bit space; OR-ing in `lo` fills the lower half. Reverse them and you get a value that's byte-swapped — wrong by roughly a factor of 256, and with the sign bit in the wrong place entirely.

The 16384.0 divisor: per the MPU6050 datasheet, the default full-scale range is $\pm 2g$, at a sensitivity of 16384 LSB per g. Dividing the raw signed value by that constant converts the raw integer into g-units. Change the configured range ($\pm 4g/\pm 8g/\pm 16g$) and this constant changes too — it's not universal, it's tied to whatever range you configured (or left at its power-on default, as here).

Lesson 11 — Motor + driver (L293D)

```
void drive(bool forward, int duty) {
    digitalWrite(IN1, forward ? HIGH : LOW);
    digitalWrite(IN2, forward ? LOW : HIGH);
    softPwmWrite(EN, duty);
}

// document in a comment:
// IN1=HIGH, IN2=HIGH means BRAKE (both motor terminals driven to the same
// polarity, shorting the motor's back-EMF through the driver) -- not
// destructive on the L293D itself, but never intentional: it's neither
// forward nor reverse and will jerk-stop the motor hard.
```

Predict-first answer: forward is `IN1=HIGH, IN2=LOW`; reverse is the opposite, `IN1=LOW, IN2=HIGH`. `IN1==IN2` (both HIGH or both LOW) ties the motor's two terminals to the same voltage rail — no potential difference across the motor, so it can't spin in either direction. Both-HIGH is a hard **brake** (motor's own back-EMF gets shorted through the driver, stopping it abruptly); both-LOW is a gentler **coast** (motor terminals float relative to each other through the driver's output stage). Neither is "forbidden" in the sense of damaging the L293D, but neither is a direction — driving both pins to the same level is a state you want to reach *deliberately* (to brake) rather than stumble into by a logic bug.

The rule that actually is inviolable here isn't about IN1/IN2 — it's the power rule from the gotcha box: never route the motor's supply through the Pi's 3.3V/5V rail. Motors draw current spikes well beyond what the Pi's regulator is designed to source, and back-EMF from the motor's coil can spike well above logic-safe voltages. The L293D's separate `VMOTOR` pin exists specifically to isolate that.

Lesson 12 — LED matrix: light one pixel

```
uint16_t rgb565(uint8_t r, uint8_t g, uint8_t b) {
    return ((r >> 3) << 11) | ((g >> 2) << 5) | (b >> 3);
}

int main() {
    int fd = open("/dev/fb1", O_RDWR);      // confirm via the detection loop -- fb0 or fb1
    varies
    auto* px = (uint16_t*) mmap(nullptr, 128,
                                PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    std::memset(px, 0, 128);
    px[0] = rgb565(255, 0, 0);              // pixel (x=0, y=0) -> index 0
}
```

Predict-first answer: RGB565 packs a 16-bit value as **5 bits red | 6 bits green | 5 bits blue**, red occupying the *most* significant bits. Red needs 5 bits of room, so it's shifted left by 11 (leaving 11 bits — 6 for green + 5 for blue — below it). Green needs 5 bits of room below it, so it's shifted left by 5. Blue sits in the bottom 5 bits with no shift. The `>> 3` / `>> 2` on each input byte discards the low bits an 8-bit channel has but a 5- or 6-bit channel doesn't — you're throwing away precision you can't represent, not adding information.

Why `px[0]` for pixel (0,0): the framebuffer is a flat array of 64 `uint16_t` values (128 bytes ÷ 2 bytes/pixel), row-major. Pixel (x,y) lives at index `y*8 + x` — for (0,0) that's `0*8+0 = 0`, the very first element.

`/dev/fb1` vs `/dev/fb0`: don't hard-code either — run the detection loop from the setup section (`for fb in /dev/fb*; ... cat ../name`) and use whichever node reports `RPi-Sense FB`. It genuinely varies by whether an HDMI display is also attached.

Lesson 13 — Joystick: read input via evdev

```
int main() {
    int fd = open("/dev/input/event0", O_RDONLY); // confirm via /proc/bus/input/devices
    struct input_event ev;
    while (read(fd, &ev, sizeof(ev)) == sizeof(ev)) {
        if (ev.type == EV_KEY && ev.value == 1) { // 1 = press
            switch (ev.code) {
                case KEY_UP:    std::cout << "UP\n";    break;
                case KEY_DOWN:  std::cout << "DOWN\n";  break;
                case KEY_LEFT:  std::cout << "LEFT\n";   break;
                case KEY_RIGHT: std::cout << "RIGHT\n";  break;
                case KEY_ENTER: std::cout << "PRESS\n";  break;
            }
        }
    }
}
```

Predict-first answer: `ev.type == EV_KEY` filters the event stream down to key-like events (as opposed to `EV_SYN`, sync markers the kernel interleaves between logical events, which you generally ignore). `ev.code` identifies *which* key (`KEY_UP`, `KEY_ENTER`, etc). `ev.value` tells you the action: `1` = pressed, `0` = released, `2` = auto-repeat (fires repeatedly while held, like a keyboard's typematic repeat) — checking specifically for `1` gives you one event per press, ignoring the repeat spam and the release.

Finding `/dev/input/eventN`: run `cat /proc/bus/input/devices` and look for the block naming "Raspberry Pi Sense HAT Joystick" — its `Handlers:` line lists the `eventN` node. This number isn't guaranteed stable across reboots (it depends on device enumeration order), so hard-coding it is fragile; detecting by name each run is the robust approach the gotcha box points at.

Lesson 14 — Environmental sensors

```
// Example shape (exact registers depend on your Sense HAT version's sensor chip --
// confirm via i2cdetect -y 1, then its datasheet; addresses/registers below are
// illustrative for an HTS221-style temperature sensor, NOT universal).
I2C sensor(0x5F);
int raw = sensor.readReg8(0x2A) | (sensor.readReg8(0x2B) << 8); // low, then high byte
double temp_c = 42.5 + raw / 100.0; // placeholder linear form -- use YOUR chip's
calibration
```

Predict-first answer: you can reuse essentially the whole `I2C` class unchanged because it wraps the generic mechanics of *any* I²C conversation — opening the bus, selecting an address, reading/writing bytes and registers. None of that logic is specific to the ADC from Lesson 8. **What changes per sensor** is purely data: the I²C address, which register numbers hold which readings, and the specific arithmetic (offsets, scale factors, sometimes calibration coefficients baked into other registers) that datasheet defines for converting the raw register bytes into a real-world unit.

This is deliberately left less filled-in than other lessons: the Sense HAT's temperature/humidity/pressure chips differ between hardware revisions (v1 vs v2 boards use different parts), so there's no single correct register map to hand you. Confirm your actual chip with `i2cdetect -y 1`, then pull its real register map and conversion formula from its datasheet before trusting any specific numbers here.

Lesson 15 — IMU orientation (RTIMULib)

```
double roll  = d.fusionPose.x() * 180.0 / std::numbers::pi;
double pitch = d.fusionPose.y() * 180.0 / std::numbers::pi;
double yaw   = d.fusionPose.z() * 180.0 / std::numbers::pi;
```

Predict-first answer: hand-rolling sensor fusion means combining accelerometer, gyroscope, and magnetometer readings — each individually noisy and each with different failure modes (accelerometer confused by linear acceleration vs. gravity; gyroscope drifts over time; magnetometer disturbed by nearby metal/electronics) — into one stable orientation estimate. That's a nontrivial numerical-methods problem (typically a Kalman filter or complementary filter) that takes real signal-processing expertise to get right, and RTIMULib has already solved it, tested it, and tuned it. Reinventing it here would spend your limited time on filter mathematics instead of the C++/Linux systems skills this plan is actually building. Lesson 10's raw reads were the right place to hand-roll something (bit assembly, two's complement) because that *is* a C++ fundamentals skill worth owning; fusion isn't in the same category.

The `.x()/y()/z()` mapping: `fusionPose` is RTIMULib's fused orientation, expressed as a 3D vector in radians. By its documented convention, `.x()` is roll (rotation around the front-to-back axis), `.y()` is pitch (nose up/down), `.z()` is yaw (compass heading) — the multiply-by-`180/pi` on each just converts radians to degrees for human-readable output, matching what the skeleton already did for roll.

A note on effort spent here

If a lesson's explanation surprised you — especially Lessons 6, 8, 10, or 12, where a shift/mask expression is doing real work — that's the signal worth sitting with. Re-derive it by hand once with a concrete example (pick actual byte values, trace the bits) rather than accepting the explanation and moving on. The bit-manipulation pattern repeats across four separate lessons in this plan; understanding it once, deeply, here, is worth more than four separate shallow passes.